data structures

mahmood

<2022-07-10 Sun 08:59:00>

1 properties of floor

Characteristic 1. for every 2 numbers m, n and every number x

$$\left\lceil \frac{\left\lceil \frac{x}{m} \right\rceil}{n} \right\rceil = \left\lceil \frac{x}{m \cdot n} \right\rceil \qquad \left\lfloor \frac{\left\lfloor \frac{x}{m} \right\rfloor}{n} \right\rfloor = \left\lfloor \frac{x}{m \cdot n} \right\rfloor$$

Example 1.0.1.

$$\left\lceil \frac{\left\lceil \frac{x}{2^i} \right\rceil}{2} \right\rceil = \left\lceil \frac{x}{2^{i+1}} \right\rceil \qquad \left\lfloor \frac{\left\lfloor \frac{x}{2^i} \right\rfloor}{2} \right\rfloor = \left\lfloor \frac{x}{2^{i+1}} \right\rfloor$$

Characteristic 2. the following is true for every normal number n and any number x

$$\lfloor x \rfloor + n = \lfloor x + n \rfloor \qquad \lceil x \rceil + n = \lceil x + n \rceil \tag{1}$$
$$n - \lfloor x \rfloor = \lceil n - x \rceil \qquad n - \lceil x \rceil = \lfloor n - x \rfloor \tag{2}$$
$$\lfloor x \rfloor \geq \lceil x \rceil - 1 \qquad \lceil x \rceil \leq \lfloor x \rfloor + 1 \tag{3}$$
$$\lfloor x \rfloor > x - 1 \qquad \lceil x \rceil < x + 1 \tag{4}$$
$$\lfloor x \rfloor \leq x \qquad \lceil x \rceil \geq x \tag{5}$$
$$\lfloor x \rfloor = n \iff n \leq x < n + 1 \qquad \lceil x \rceil = n \iff n - 1 < x \leq n \tag{6}$$

2 order of growth

Definition 2.0.1. the order of growth of an is an approximation of the time required to run a computer program as the input size increases, the order of growth ignores the constant factor needed for fixed operations and focuses instead on the operations that increase proportional to input size
an order of growth is a set of functions whose asymptotic growth behavior is considered equivalent. for example, $2n$, $100n$ and $n + 1$ belong to the same order of growth, which is written $O(n)$ in big-oh notation and often called linear because every function in the set grows linearly with n

3 asymptotic notations

notation	name	formal definition	limit definition
$f(n) = O(g(n))$	Big Oh	$\exists k > 0\ \exists n_0\ \forall n > n_0 : \lvert f(n) \rvert \leq k \cdot g(n)$	$\lim_{n\to\infty} \frac{\lvert f(n) \rvert}{g(n)} = c,$
$f(n) = \Omega(g(n))$	Big Omega	$\exists k > 0\ \exists n_0\ \forall n > n_0 : \lvert f(n) \rvert \geq k \cdot g(n)$	$\lim_{n\to\infty} \frac{g(n)}{f(n)} = c,$ v
$f(n) = \Theta(g(n))$	Big Theta	$\exists k_1 > 0\ \exists k_2 > 0\ \exists n_0\ \forall n > n_0 : k_1 \cdot g(n) \leq \lvert f(n) \rvert \leq k_2 \cdot g(n)$	$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c,$ v

3.1 Big O

3.2 Big Theta

3.3 Big Omega

3.4 time complexity

4 recursive functions

consider factorial

```
def fac(n):
  if n == 1: return 1
  else: return fac(n-1) * n
```

4.1 recurrence relation

to analyze the runtime of such a function we use a mathematical concept called **Recurrence relation**
a recurrence defines $T(n)$ in terms of T for smaller values
example: $T(n) = T(n-1) + 1$, here $T(n)$ is defined in terms of $T(n-1)$

4.1.1 initial condition

a recurrence relation must have **initial conditions**, initial conditions are values of the recurrence for small values of n
the values of $T(0), T(1)$ are usually sufficient as initial conditions

4.1.2 closed form

to solve a recurrence, we find a **closed form** for it, a closed form for $T(n)$ is an equation that defines $T(n)$ using an expression that does **not** involve T
there is no single method that works for all, we check for patterns and use substitution
to get a better understanding, we look at examples

Example 4.1.1. consider the following function that returns the factorial of a given number

```
def fac(n):
  if n == 1: return 1
  else: return n * fac(n-1)
print([fac(i) for i in range(1,10)])
```

for `fac(n)`, how many times is `fac` called?

$$T(n) \leq T(n-1) + C_1$$

C_1 represents the total time of all the operations that take constant time that the function does on every iteration
now this is obviously not what we wanna arrive at, we need to arrive at an expression that doesnt have T in it
we can substitute in $n-1$ because T is a function that takes any positive integer

$$T(n) \leq \underbrace{T(n-2) + C_1}_{T(n-1)} + C_1$$
$$T(n) \leq T(n-3) + 3C_1$$

now you probably already see the pattern here, on every iteration the function takes constant time C_1 to run, until it reaches the last iteration k at which point it we would've k times C_1, e.g.:

$$T(n) \leq T(n-k) + kC_1$$

now we know that the total number of times the function calls itself depends on the initial value of n, in this case that number would be $n-1$, as in, this function calls itself $n-1$ times
we substitute $k = n-1$ to get the total time this function takes to run

$$\begin{aligned} T(n) &\leq T(n-(n-1)) + (n-1)C_1 \\ &\leq T(1) + (n-1)C_1 \\ &\leq C_1 + (n-1)C_1 && \text{because } C_1 \geq T(1) \\ &\leq nC_1 \end{aligned}$$

so we conclude that

$$\Theta(n) = nC_1 \implies \Theta(n) = n$$

Example 4.1.2. consider the following function that returns the maximum value in a given array

```
def mymax(arr, l, r):
  if l == r:
    return arr[l]
  return max(arr[l], mymax(arr, l+1, r))
arr = [3, 10, 81, 30, 58, 0, -10]
print(mymax(arr, 0, len(arr)-1))
```

we need to find a recurrence for this function so we can analyze its time complexity
first thing we notice is on every iteration we have a constant time (excluding the recursive call), we call it C
we can also notice that the variable "r" never changes, so our recurrence should only depends on l
now we can guess that the recurrence equation is:

$$T(l) = T(l-1) + C$$

you might be confused (hopefully not) that we wrote $l-1$ even though in the recursive call its $l+1$, this is because we dont care about the value of the variable itself but rather about how many times the variable causes the function to call itself
now if we had written $l+1$ this would suggest that given the value l the function runs $l+1$ times which is not correct
and so this equals to $\Theta(n)$ (using the same proof as the previous problem)

Example 4.1.3. consider the following function that returns the fibonacci number at the given index

```python
def fib(n):
  if n in [1, 2]: return 1
  else: return fib(n-1) + fib(n-2)
  print([fib(i) for i in range(1,10)])
```

for `fib(n)`, how many times is `fib` called?

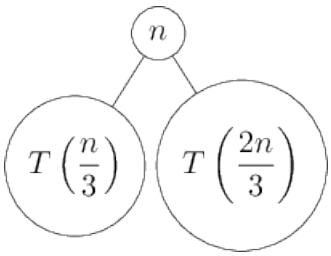
$$
\begin{aligned}
T(1) &= 1 \\
T(2) &= 1 \\
T(3) &= T(1) + T(2) + 1 = 1 + 1 + 1 = 3 \\
T(4) &= T(3) + T(2) + 1 = 3 + 1 + 1 = 5 \\
T(5) &= T(4) + T(3) + 1 = 5 + 3 + 1 = 9 \\
T(n) &= T(n-1) + T(n-2) + 1 = 2^n + 1
\end{aligned}
$$

4.2 recursion tree

Example 4.2.1. consider the following recurrence relation

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n$$

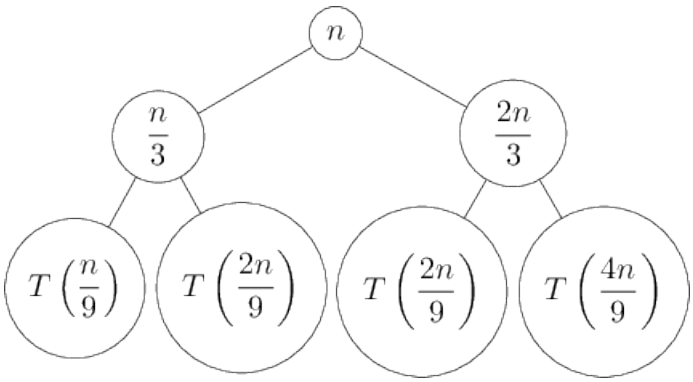
a note to keep in mind is that the sum of all the nodes of the tree must always be equal to $T(n)$
with that in mind, the first step would be:



if we sum all the nodes we can see that indeed $T(n) = T\left(\frac{2n}{3}\right) + T\left(\frac{n}{3}\right) + n$
for the next step we need to write $T\left(\frac{2n}{3}\right)$ and $T\left(\frac{n}{3}\right)$ in terms of time complexity for smaller values of n so we can know what the next row of nodes would be

$$
\begin{aligned}
T\left(\frac{2n}{3}\right) &= T\left(\frac{2 \cdot \frac{2n}{3}}{3}\right) + T\left(\frac{\frac{2n}{3}}{3}\right) + \frac{2n}{3} = T\left(\frac{4n}{9}\right) + T\left(\frac{2n}{9}\right) + \frac{2n}{3} \\
T\left(\frac{n}{3}\right) &= T\left(\frac{2 \cdot \frac{n}{3}}{3}\right) + T\left(\frac{\frac{n}{3}}{3}\right) + \frac{n}{3} = T\left(\frac{2n}{9}\right) + T\left(\frac{n}{9}\right) + \frac{n}{3}
\end{aligned}
$$

according to this, the tree with the new nodes would be:

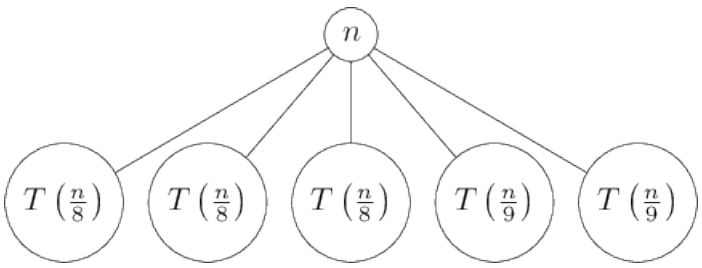


we define a **full row** as a row that is full of nodes, in the previous tree the rows $1, 2, 3$ are full, we might notice that the sum of all the nodes of a full row is n
let y be the last full row
because the leftmost node is always the smallest in its row, for a row to be full the leftmost node has to be greater or equal to 1 ($\frac{n}{3^y} \geq 1$ in this case)
if $\frac{n}{3^y} \geq 1$ is true then the leftmost node in the row $y+1$ whose value is $\frac{n}{3^y}$ appears in the tree (because nodes with values greater or equal to 1 do appear in the tree), and so the row $y+1$ is full, which results in a contradiction because y is the last full row
therefore it must be that $\frac{n}{3^y} < 1 \Rightarrow n < 3^y \Rightarrow y > \log_3 n$
therefore the tree contains atleast $\log_3 n$ full rows, and because the sum of every full row is n we get:
$T(n) =$ sum of all nodes $=$ sum of nodes in full rows $+$ sum of nodes in unfull rows $\geq$ sum of nodes in full rows $\geq n \cdot \log_3 n = \Theta(n \log n)$
and therefore $T(n) = \Omega(n \log n)$
let x be the last row in the tree, the value of the rightmost node in row x would be $\left(\frac{2}{3}\right)^{x-1} \cdot n$
if this value is less than 1 then this node doesnt appear in the tree and all the other nodes in row x that have a smaller value also dont exist which gives us a contradiction because x is the last row in the tree
therefore we get $\left(\frac{2}{3}\right)^{x-1} n \geq 1 \Rightarrow n \geq \left(\frac{3}{2}\right)^{x-1} \Rightarrow \log_{3/2} n \geq x - 1 \Rightarrow x \leq 1 + \log_{3/2} n$
and therefore in the tree there is at most $1 + \log_{3/2} n$ rows, and since the sum of every row in the tree is n we get:
$T(n) =$ sum of all nodes $=$ sum of all nodes in all rows $\leq n \cdot$ number of rows in the tree $\leq n \cdot (1 + \log_{3/2} n) = \Theta(n \log n)$
and therefore $T(n) = O(n \log n)$
in summery, we proved that $T(n) = \Theta(n \log n)$

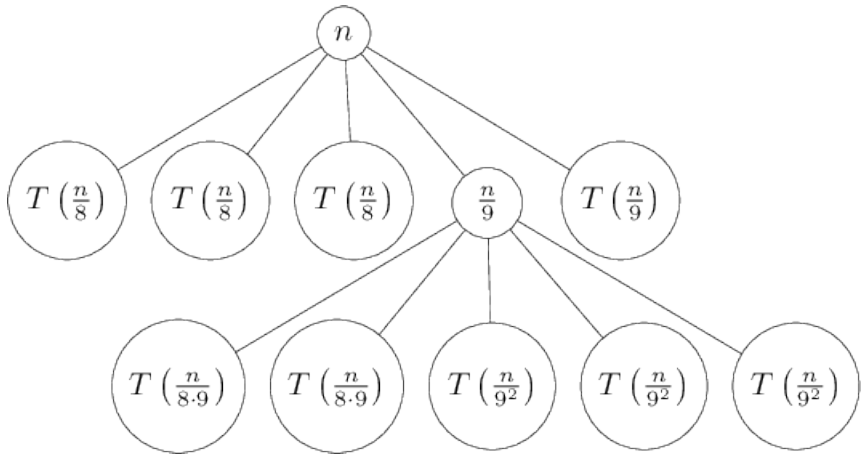
Example 4.2.2. consider the following recurrence relation

$$T(n) = 2T\left(\frac{n}{8}\right) + 3T\left(\frac{n}{9}\right) + n$$

we cant determine Θ using the Master Theorem, but we can using a recursion tree
the first step would be:



$$T\left(\frac{n}{8}\right) = 2T\left(\frac{n}{8^2}\right) + 3T\left(\frac{n}{8 \cdot 9}\right) + \frac{n}{8}$$
$$T\left(\frac{n}{9}\right) = 2T\left(\frac{n}{8 \cdot 9}\right) + 3T\left(\frac{n}{9^2}\right) + \frac{n}{9}$$



the sum of the nodes in level 1 is n, in level 2 its $n\left(\frac{2}{8}+\frac{3}{9}\right)$, in level 3 its $n\left(\frac{2}{8}+\frac{3}{9}\right)^2$, and so in level i it is $n\left(\frac{2}{8}+\frac{3}{9}\right)^{i-1}$
we denote by x the last level

$$\begin{aligned}
T(n) &= \text{sum of all nodes} \\
&= \text{sum of all nodes in all levels} \\
&\leq n + n\left(\frac{2}{8}+\frac{3}{9}\right) + n\left(\frac{2}{8}+\frac{3}{9}\right)^2 + \cdots + n\left(\frac{2}{8}+\frac{3}{9}\right)^{x-1} \\
&\leq n\left(\left(\frac{2}{8}+\frac{3}{9}\right) + \left(\frac{2}{8}+\frac{3}{9}\right)^2 + \cdots + \left(\frac{2}{8}+\frac{3}{9}\right)^{x-1}\right) \\
&= n \cdot \frac{1}{1-\left(\frac{2}{8}+\frac{3}{9}\right)} \\
&= \Theta(n)
\end{aligned}$$

note the sum was found using
and so we have shown that $T(n) = O(n)$

$$T(n) = \text{sum of all nodes in the tree} \geq \text{root} = n = \Theta(n)$$

and so we have shown that $T(n) = \Omega(n)$
and so $T(n) = \Theta(n)$

4.3 Master Theorem

4.3.1 dividing functions

for a given recurrence relation of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n) \text{ where } a \geq 1, b > 1$$

we compare $f(n)$ with $n^{\log_b a}$ and check which one dominates, the possible outcomes are:

$$\exists \epsilon > 0 \mid f(n) = O\left(n^{\log_b(a)-\epsilon}\right) \implies T(n) = \Theta\left(n^{log_b a}\right) \tag{7}$$

$$f(n) = \Theta\left(n^{\log_b a}\right) \implies T(n) = \Theta\left(n^{\log_b a}\log n\right) \tag{8}$$

$$\exists \epsilon > 0 \mid f(n) = \Omega\left(n^{\log_b(a)+\epsilon}\right) \text{ and } 0 < c < 1 \mid af\left(\frac{n}{b}\right) < cf(n) \implies T(n) = \Theta(f(n)) \tag{9}$$

Example 4.3.1. example of the first possibility

$$T(n) = 8T\left(\frac{n}{2}\right) + 1000n^2$$
$$a = 8, b = 2, f(n) = 1000n^2$$

the conditions of the first possibility are met:

$$n^{\log_b a} = n^{\log_2 8} = n^3 = O\left(1000n^2\right)$$

therefore we get:

$$T(n) = \Theta(n^3)$$

Example 4.3.2. example of the second possibility

$$T(n) = 2T\left(\frac{n}{2}\right) + 10n$$
$$a = 2, b = 2, f(n) = 10n$$

the conditions of the second possibility are met:

$$n^{\log_b a} = n^{\log_2 2} = n^1 = \Theta(10n)$$

we get:

$$T(n) = \Theta(n \log n)$$

Example 4.3.3. example of the third possibility

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$
$$a = 2, b = 2, f(n) = n^2$$

the conditions of the third possibility are met:

$$n^{\log_b a} = n^{\log_2 2} = n^1 = \Omega(n^2)$$

we get:

$$T(n) = \Theta\left(n^2\right)$$

4.3.2 decreasing functions

The master theorem can be used to solve recurrences of the form $T(n) = aT(n-b) + f(n)$, where $a \geq 1$ and $b > 0$ and $f(n)$ is *Asymptotically positive.* (Asymptotically positive means that the function is positive for all sufficiently large n.) This recurrence describes an algorithm that divides a problem of size n into sub problems, each of size $n - b$, and solves them recursively.
the theorem is as follows:
if $T(n) = aT(n-b) + f(n)$, where $a \geq 1$, $b > 0$, $f(n) = O(n^k)$ and $k \geq 0$
we consider 3 cases:
case 1, if $a = 1$

$$T(n) = O(n \cdot f(n)) \text{ or } O\left(n^{k+1}\right)$$

examples:

$$T(n) = T(n-1) + 1 \Rightarrow O(n) \tag{10}$$
$$T(n) = T(n-1) + n \Rightarrow O(n^2) \tag{11}$$
$$T(n) = T(n-1) + \log n \Rightarrow O(n \log n) \tag{12}$$

case 2, if $a > 1$

$$T(n) = O(a^{\frac{n}{b}} \cdot f(n)) \text{ or } O(a^{\frac{n}{b}} \cdot n^k)$$

examples:

$$T(n) = 2T(n-1) + 1 \Rightarrow O(2^n) \tag{13}$$
$$T(n) = 3T(n-1) + 1 \Rightarrow O(3^n) \tag{14}$$
$$T(n) = 2T(n-1) + n \Rightarrow O(n \cdot 2^n) \tag{15}$$

case 3, if $a < 1$

$$T(n) = O(f(n)) \text{ or } O\left(n^k\right)$$

5 data structure

data structure is a collection of data and operations on said data, the data is represented as objects, and each object `x` has:

- `key(x)`: a key that is used to identify the object (usually a number that uniquely identifies the object)
- `info(x)` or `value(x)`: contains the data needed to store for the object (usually without the key)

for example, if `x` is a person, `key(x)` could be the person's ID, `value(x)` would be information on said person, like age, height, weight etc..

5.1 List

a **list** is a dynamically sized (often automatically resized) *list of objects* that stores as many objects as necessary, each type of list has its own method of dynamic storage

5.1.1 linked list

characteristics

- is the list **sorted**? the nodes are sorted by their `key` (usually in an ascending order)
- is the list **doubly/singly linked**? in a doubly linked list each node points to the next and previous node whereas in a singly linked list a node only points to the next
- does the list have a **tail**? a pointer `tail(x)` points to the last node in a linked list
- is the list **circular**? in a circular linked list the last node points to the first whereas in a non-circular linked list it points to NULL (nothing)

functions

- `insert(x)`: inserts `x`
- `search(k)`: returns the node whose key is `k`
- `isEmpty()`: returns whether list is empty
- `delete(x)`: given the node `x`, removes it from the list

implementation (c++)

Node

```
#include <cassert>

template <typename T> class Node {
private:
  T data;
  Node<T>* next;

public:
  Node(T data, Node<T>* next) {
    this->data = data;
    this->next = next;
  }
  T get_data() {
    return this->data;
  }
  Node<T>* get_next() {
    return this->next;
  }
  void set_next(Node<T>* next) {
    assert(this != next);
    this->next = next;
  }
  bool has_next() {
    return this->next != nullptr;
```

```
  }
};
```

DoublyNode

```
template <typename T> class DoublyNode {
private:
  DoublyNode<T>* left = nullptr;
  DoublyNode<T>* right = nullptr;
  T data;

public:
  DoublyNode(T data) {
    this->data = data;
  }
  ~DoublyNode() {
    delete left;
    delete right;
  }
  T get_data() {
    return this->data;
  }
  void set_left(DoublyNode<T> node) {
    this->left = node;
    if (this->left != nullptr) {
      this->left->parent = this;
    }
  }
  void set_right(DoublyNode<T> node) {
    this->right = node;
    if (this->right != nullptr) {
      this->right->parent = this;
    }
  }
  DoublyNode<T> get_right() {
    return this->right;
  }
  DoublyNode<T> get_left() {
    return this->left;
  }
  bool has_right() {
    return this->right != nullptr;
  }
  bool has_left() {
    return this->left != nullptr;
  }
};
```

LinkedList

```
template <typename T> class LinkedList {
private:
  Node<T>* head = nullptr;

public:
  ~LinkedList() {
    Node<T>* n = this->head;
```

```
    while (n) {
      Node<T>* next = n->get_next();
      delete n;
      n = next;
    }
  }
  Node<T>* get_head() {
    return this->head;
  }
  void set_head(Node<T>* n) {
    this->head = n;
  }
  LinkedList* add(T data) {
    return this->add(new Node<T>(data, nullptr));
  }
  LinkedList* add(Node<T>* n) {
    if (this->is_empty()) {
      this->head = n;
    } else {
      this->get_tail()->set_next(n);
    }
    return this;
  }
  Node<T>* get_tail() {
    if (this->is_empty())
      return nullptr;
    Node<T>* n;
    for (n = this->head; n->get_next() != nullptr; n = n->get_next());
    return n;
  }
  bool is_empty() {
    return this->head == nullptr;
  }
  Node<T>* get_node(int index) {
    Node<T>* n = head;
    for (int i = 0; i < index; ++i) {
      n = n->get_next();
    }
    return n;
  }
  int size() {
    Node<T>* n;
    int i = 0;
    for (n = this->head; n != nullptr; n = n->get_next()) {
      ++i;
    }
    return i;
  }
};
```

DoublyLinkedList

```
to be implemented
```

LinkedListWithTail

```
to be implemented
```

CircularLinkedList

```
to be implemented
```

5.1.2 array list

an array list contains a fixed-size array initialized to a constant length, when the array gets filled we create a new array whose size is $n \times c$ where n is a constant (usually 2), and c is the previous size of the array, and we copy all the objects to the new array and then we have more space to store more objects

ArrayList

```
#include <cassert>

#define INITIAL_SIZE 3
#define SCALING_FACTOR 2

template <typename T> class ArrayList {
private:
  int actualSize; int idx = 0;
  T* arr;
  T* allocateMemory(int numOfObjects) {
    T* arr = new T[numOfObjects];
    this->actualSize = numOfObjects;
    return arr;
  }
  void resize() {
    T* newArr = allocateMemory(this->actualSize * SCALING_FACTOR);
    for (int i = 0; i < idx; ++i) {
      newArr[i] = arr[i];
    }
    delete[] this->arr;
    this->arr = newArr;
  }

public:
  ArrayList() {
    this->arr = this->allocateMemory(INITIAL_SIZE);
  }
  ~ArrayList() {
    delete[] this->arr;
  }
  ArrayList* add(T val) {
    if (idx == this->actualSize)
      this->resize();
    arr[idx++] = val;
    return this;
  }
  T get(int index) {
    return arr[index];
  }
  int size() {
    return this->idx;
  }
  void remove(int index) {
    assert(index < this->idx);
    if (index < this->idx-1) {
```

```
      for (int i = index; i < this->idx-1; ++i) {
        arr[i] = arr[i+1];
      }
    }
    this->idx--;
  }
  void insert(int index, T value) {
    assert(index < idx);
    if (idx == this->actualSize-1)
      this->resize();
    for (int i = idx; i > index; --i) {
      arr[i] = arr[i-1];
    }
    arr[index] = value;
    idx++;
  }
};
```

5.2 Queue

a Queue is a linear data structure in which the operations are performed based on FIFO principle

5.2.1 FIFO

Definition 5.2.1. FIFO stands for **first in first out**, which means the first entity that enters would be the first one to exit

5.2.2 Circular Queue

Definition 5.2.2. a **Circular Queue** is a Queue in which the last element is connected back to the first element to make a circle.

5.3 Tree

5.3.1 binary tree

a **binary tree** is a data structure which stores data in the shape of a downwards-growing tree

height the height of the tree is the number of the node's in the longest path downwards

node binary tree's store data as **nodes**, meaning each item in a binary tree is called a node
each node `x` has the following fields

- `key(x)`: a key that's used to identify the node (like every data structure)
- `info(x)`: contains the data we need to store (like every data structure)
- `left(x)`: a pointer to the left child of `x`

- `right(x)`: a pointer to the right child

- `parent(x)` (optional): a pointer to the parent of `x`

every node in a binary tree has from 0 to 2 children

height the **height** of a node `x` is defined as the **height** of the subtree of said node, denoted by T_x

root

Definition 5.3.1. the tree itself `T` (not for every node) contains a pointer `root(T)` that points to the root node of the tree (the first and topmost node)

leaf

Definition 5.3.2. a node that doesnt have children is called a **leaf**

internal node

Definition 5.3.3. a node that has atleast one child is called an **internal node**

sibling

Definition 5.3.4. 2 nodes are considered siblings if they share the same parent node

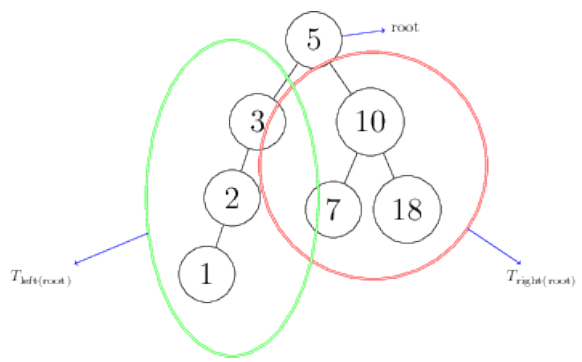
ancestor

Definition 5.3.5. an **ancestor** of a given node is a node that is at an upper level of it

descendant

Definition 5.3.6. a **descendant** of a given node is a node that is at a lower level of the given node

subtree for some binary tree `T` and a node `x` of said tree we denote T_x as the **subtree** whose root is `x`, and the left **subtree** of `x` is the tree whose root is the left child of `x` denoted by $T_{\text{left}(x)}$ and the right subtree of `x` is the tree whose root is the right child of `x` denoted by $T_{\text{right}(x)}$



the height of this tree is 4

complete tree

Definition 5.3.7. a **complete** binary tree is a binary tree in which every node has 2 or 0 children, and the leaves are all on at the same level

almost complete tree

Definition 5.3.8. an **almost complete** binary tree is a binary tree from which 0 or more nodes have been deleted continuously from the right part of the last level of nodes

Characteristic 3. the nodes in the last level in an almost complete binary tree have to appear continuously from left to right

Characteristic 4. every complete tree is an almost complete tree but not every almost complete tree is a complete tree

Characteristic 5. every level contains 2 times as many nodes as the level before it, except perhaps for the last level

Characteristic 6. the height of a tree is $\lfloor \log_2 n \rfloor$

Characteristic 7. the number of nodes in a tree is $\left\lceil \frac{n}{2} \right\rceil$

Characteristic 8. the number of nodes at height i is $\left\lceil \frac{n}{2i+1} \right\rceil$

breadth-first search **breadth-first search** or **BFS** for short, explores each level before moving onto the next one
extra memory, usually a queue, is needed to keep track of the child nodes that were encountered but not yet explored.
`./Animated_BFS.gif`

depth-first search **depth-first search** or **DFS** for short, consists of exploring branches as far as possible before backtracking and exploring branches of other nodes
this is the algorithm we'll be using because its simpler
`./bfs_dfs.gif`

inorder traversal left, parent, right

```
Algorithm inorder(tree)
  1. go to the left child, i.e., call inorder(left-child), print it
  2. go back to the parent, print it
  3. go to the right child, i.e., call inorder(right-child), print it
```

note
note that before printing a child will always call `inorder(left-child)`, so the first node to print would be the leftmost node in the deepest level

preorder traversal parent, left, right

```
Algorithm preorder(tree)
  1. visit the parent, print it
  2. go to the left child, i.e., call preorder(left-child), print it
  3. go to the right child, i.e., call preorder(right-child), print it
```

postorder traversal left, right, parent

```
Algorithm postorder(tree)
  1. go to the left child, i.e., call postorder(left-child), print it
  2. go to the right child, i.e., call postorder(right-child), print it
  3. visit the parent, print it
```

BinaryTreeNode

```
#include <cstdlib>

template <typename T> class BinaryTree; /* forward declare the class instead of including it
    */

template <typename T> class BinaryTreeNode {
private:
  BinaryTreeNode<T>* parent = nullptr;
  BinaryTreeNode<T>* right = nullptr;
  BinaryTreeNode<T>* left = nullptr;
  T value;
  int key;

public:
  BinaryTreeNode(int key, T value) {
    this->key = key;
    this->value = value;
  }
  BinaryTreeNode(int key) {
    this->key = key;
    this->value = key;
  }
  ~BinaryTreeNode() {
    delete this->right;
    delete this->left;
  }
  void set_parent(BinaryTreeNode<T>* node) {
    this->parent = node;
  }
  void set_right(BinaryTreeNode<T>* node) {
    this->right = node;
    if (node != nullptr)
      node->parent = this;
  }
  void set_left(BinaryTreeNode<T>* node) {
    this->left = node;
```

```
    if (node != nullptr)
      node->parent = this;
  }
  BinaryTreeNode<T>* get_parent() {
    return this->parent;
  }
  BinaryTreeNode<T>* get_right() {
    return this->right;
  }
  BinaryTreeNode<T>* get_left() {
    return this->left;
  }
  bool has_right() {
    return this->right != nullptr;
  }
  bool has_left() {
    return this->left != nullptr;
  }
  bool has_parent() {
    return this->parent != nullptr;
  }
  T get_value() {
    return this->value;
  }
  int get_key() {
    return this->key;
  }
  bool is_leaf() {
    return !(this->has_left() || this->has_right());
  }
  bool is_right_child() {
    return this->parent != nullptr && this->parent->get_right() == this;
  }
  bool is_left_child() {
    return this->parent != nullptr && this->parent->get_left() == this;
  }
  int height() {
    int right_height = this->has_left() ? this->get_left()->height() : 0;
    int left_height = this->has_right() ? this->get_right()->height() : 0;
    if (left_height > right_height) {
      return 1 + left_height;
    }
    return 1 + right_height;
  }
  BinaryTreeNode<T>* find(int key) {
    if (this->key == key) {
      return this;
    }
    BinaryTreeNode<T>* n_left = this->has_left() ? this->get_left()->find(key) : nullptr;
    BinaryTreeNode<T>* n_right = this->has_right() ? this->get_right()->find(key) : nullptr;
    return n_left != nullptr ? n_left : n_right;
  }
  int balance_factor() {
    if (this->has_left()) {
      if (this->has_right()) {
        return this->get_left()->height() - this->get_right()->height();
```

```
    } else {
      return this->get_left()->height();
    }
  }
  if (this->has_right()) {
    return 0 - this->get_right()->height();
  }
  return 0;
}
bool is_balanced() {
  bool is_left_balanced = this->has_left() ? this->get_left()->is_balanced() : true;
  bool is_right_balanced = this->has_right() ? this->get_right()->is_balanced() : true;
  return !this->is_problematic() && is_left_balanced && is_right_balanced;
}
bool is_problematic() {
  return abs(this->balance_factor()) > 1;
}
BinaryTreeNode<T>* find_problematic_node() {
  if (this->is_problematic())
    return this;
  BinaryTreeNode<T>* p_left = nullptr;
  BinaryTreeNode<T>* p_right = nullptr;
  if (this->has_left())
    p_left = this->get_left()->find_problematic_node();
  if (this->has_right())
    p_right = this->get_right()->find_problematic_node();
  return p_left == nullptr ? p_right : p_left;
}
bool is_complete(BinaryTreeNode<T>* root) {
  if (this->is_leaf())
    return this->distance_to(root) == root->height();
  if (!this->has_right() || !this->has_left())
    return false;
  return this->right->is_complete(root) && this->left->is_complete(root);
}
int distance_to(BinaryTreeNode<T>* other) {
  if (other->height() > this->height()) {
    return other->find_path(this)->height();
  }
  return this->find_path(other)->height();
}
BinaryTreeNode<T>* find_path(BinaryTreeNode<T>* other) {
  if (this == other) {
    return new BinaryTreeNode<T>(other->key, other->value);
  } else {
    if (this->has_left()) {
      BinaryTreeNode<T>* n_left = this->get_left()->find_path(other);
      if (n_left != nullptr) {
        BinaryTreeNode<T>* node = new BinaryTreeNode<T>(this->key, this->value);
        node->set_left(n_left);
        return node;
      }
    }
    if (this->has_right()) {
      BinaryTreeNode<T>* n_right = this->get_right()->find_path(other);
      if (n_right != nullptr) {
```

```
        BinaryTreeNode<T>* node = new BinaryTreeNode<T>(this->key, this->value);
        node->set_right(n_right);
        return node;
      }
    }
  }
  return nullptr;
}
BinaryTreeNode<T>* find_rightmost_leaf() {
  if (this->is_leaf() && this->level() == this->find_root()->height()) {
    return this;
  }
  BinaryTreeNode<T>* from_right = this->has_right() ? this->get_right()->
      find_rightmost_leaf() : nullptr;
  if (from_right != nullptr)
    return from_right;
  return this->has_left() ? this->get_left()->find_rightmost_leaf() : nullptr;
}
BinaryTreeNode<T>* find_leftmost_leaf() {
  if (this->is_leaf() && this->level() == this->find_root()->height() - 1) {
    return this;
  }
  BinaryTreeNode<T>* from_left = this->has_left() ? this->get_left()->find_leftmost_leaf()
      : nullptr;
  if (from_left != nullptr)
    return from_left;
  return this->has_right() ? this->get_right()->find_leftmost_leaf() : nullptr;
}
bool has_right_and_left() {
  return this->has_left() && this->has_right();
}
bool is_right() {
  return this->parent->get_right() == this;
}
bool is_left() {
  return this->parent->get_left() == this;
}
BinaryTreeNode<T>* get_right_uncle() {
  return this->parent->parent->right;
}
int level() {
  int lvl = 1;
  BinaryTreeNode<T>* n = this;
  while (n->has_parent()) {
    lvl++;
    n = n->get_parent();
  }
  return lvl;
}
BinaryTreeNode<T>* find_root() {
  BinaryTreeNode<T>* n = this;
  while (n->has_parent()) {
    n = n->get_parent();
  }
  return n;
}
```

```
  void print() {
    if (this->has_left())
      this->left->print();
    std::cout << this->key << " ";
    if (this->has_right())
      this->right->print();
  }
};
```

BinaryTree

```


template <typename T> class BinaryTree {
private:
  BinaryTreeNode<T>* root = nullptr;
public:
  BinaryTree() {}
  BinaryTree(BinaryTreeNode<T>* root) {
    this->root = root;
  }
  ~BinaryTree() {
    delete this->root;
  }
  BinaryTreeNode<T>* get_root() {
    return this->root;
  }
  void set_root(BinaryTreeNode<T>* node) {
    this->root = node;
  }
  bool is_empty() {
    return this->root == nullptr;
  }
  int size() {
    BinaryTreeNode<T>* n;
    int i = 0;
    for (n = this->get_root(); n != nullptr; n = n->get_right()) {
      ++i;
    }
    return i;
  }
  int height() {
    if (this->is_empty())
      return 0;
    return this->root->height();
  }
  bool is_balanced() {
    if (this->is_empty()) {
      return true;
    }
    return this->get_root()->is_balanced();
  }
  BinaryTreeNode<T>* find_problematic_node() {
    return this->root->find_problematic_node();
  }
  BinaryTreeNode<T>* find(int key) {
    return this->root->find(key);
```

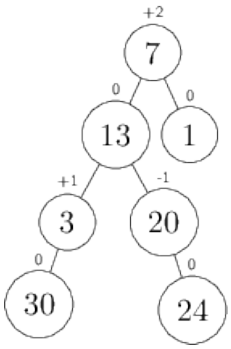
```
  }
  /* inorder print */
  void print() {
    if (!this->is_empty()) this->get_root()->print();
    std::cout << "\n";
  }
  void empty() {
    delete this->root;
    this->root = nullptr;
  }
  BinaryTree<T>* insert_right(BinaryTreeNode<T>* node) {
    if (this->is_empty()) {
      this->set_root(node);
      return this;
    }
    BinaryTreeNode<T>* n = this->get_root();
    while (n->has_right()) {
      n = n->get_right();
    }
    n->set_right(node);
    return this;
  }
  BinaryTree<T>* insert_left(BinaryTreeNode<T>* node) {
    if (this->is_empty()) {
      this->set_root(node);
      return this;
    }
    BinaryTreeNode<T>* n = this->get_root();
    while (n->has_left()) {
      n = n->get_left();
    }
    n->set_left(node);
    return this;
  }
  BinaryTree<T>* insert_right(int key) {
    return this->insert_right(new BinaryTreeNode<int>(key));
  }
  BinaryTree<T>* insert_left(int key) {
    return this->insert_left(new BinaryTreeNode<int>(key));
  }
  bool is_complete() {
    return this->is_empty() || this->root->is_complete(this->root);
  }
  /* note that this function gets the rightmost leaf in the last level */
  BinaryTreeNode<T>* find_rightmost_leaf() {
    return this->root->find_rightmost_leaf();
  }
  /* note that this function gets the leftmost leaf in the second-to-last level */
  BinaryTreeNode<T>* find_leftmost_leaf() {
    return this->root->find_leftmost_leaf();
  }
};
```

balance factor the **balance factor** of a node `x`, denoted by `balance(x)` or `BF(x)`, is defined as the height of the subtree of the left child of `x`, denoted by $T_{\text{left}(x)}$ minus the height of the subtree of the right child of `x`

denoted by $T_{\text{right}(x)}$, in short:

$$BF(X) = \text{height}(T_{\text{left}(x)}) - \text{height}(T_{\text{right}(x)})$$

see for example this tree, the balance factor of each node is written above it

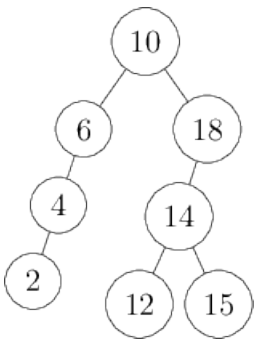


5.3.2 binary search tree

a **binary search tree** is a binary tree that has the following restrictions:

- the keys of all the nodes in the left subtree of any given node `x` is smaller than the key of `x`
- the keys of all the nodes in the right side of the tree are bigger or equal to the key of `x`

for example the following tree is a binary search tree:



insertion into BST a new key is always inserted as a leaf. we start from the root, making our way downwards by comparing the new key to the key of the current node to decide which child to move to on each iteration, until we hit a leaf node, then we add the new node as its child

BinarySearchTree

```
template <typename T> class BinarySearchTree : public BinaryTree<T> {
public:
  BinarySearchTree() {}
  BinarySearchTree(BinaryTreeNode<T>* node) : BinaryTree<T>(node) {}
  virtual BinarySearchTree<T>* insert(BinaryTreeNode<T>* node) {
    node->set_right(nullptr);
    node->set_left(nullptr);
    if (this->is_empty()) {
      BinaryTree<T>::set_root(node);
```

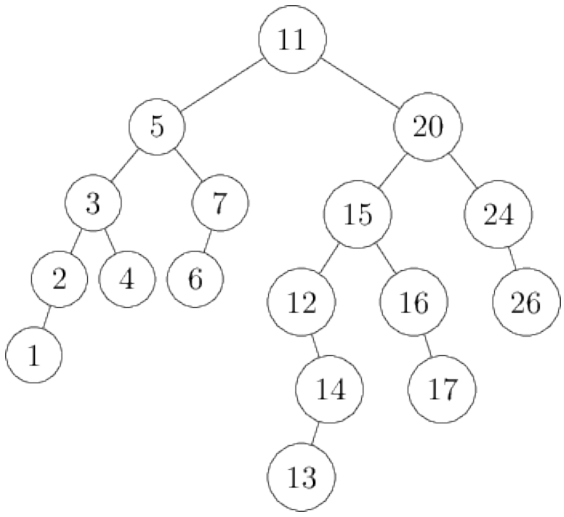
```
  } else {
    BinaryTreeNode<T>* n = this->get_root();
    while (true) {
      if (node->get_key() < n->get_key()) {
        if (!n->has_left()) {
          n->set_left(node);
          node->set_parent(n);
          break;
        }
        n = n->get_left();
      } else {
        if (!n->has_right()) {
          n->set_right(node);
          node->set_parent(n);
          break;
        }
        n = n->get_right();
      }
    }
  }
  return this;
}
BinarySearchTree<T>* insert(int key, T value) {
  return this->insert(new BinaryTreeNode<T>(key, value));
}
BinarySearchTree<T>* insert(int value) {
  return this->insert(new BinaryTreeNode<T>(value));
}
BinarySearchTree<T>* insert(int* values, unsigned int size) {
  for (int i = 0; i < size; ++i)
    this->insert(new BinaryTreeNode<T>(values[i]));
  return this;
}
/* this needs to be implemented asap */
void remove(BinaryTreeNode<T>* node) {
  /* in case that node is the only one in the tree */
  if (this->get_root()->is_leaf() && this->get_root() == node) {
    this->set_root(nullptr);
    return;
  }
  /* in case the node has no children */
  BinaryTreeNode<T>* parent = node->get_parent();
  if (node->is_leaf()) {
    if (node->is_right_child()) {
      parent->set_right(nullptr);
    } else {
      parent->set_left(nullptr);
    }
  }
}
void rotate_right(BinaryTreeNode<T>* node) {
  BinaryTreeNode<T>* previous_left = node->get_left();
  if (node->is_left_child()) {
    node->get_parent()->set_left(previous_left);
  } else if (node->is_right_child()) {
    node->get_parent()->set_right(previous_left);
```

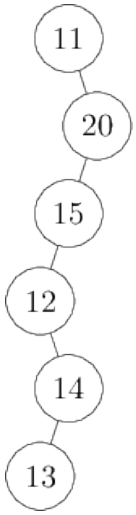
```
    } else if (node == this->get_root()) {
      this->set_root(previous_left);
      previous_left->set_parent(nullptr);
    }
    node->set_parent(previous_left);
    node->set_left(previous_left->get_right());
    previous_left->set_right(node);
  }
  void rotate_left(BinaryTreeNode<T>* node) {
    BinaryTreeNode<T>* previous_right = node->get_right();
    if (node->is_right_child()) {
      node->get_parent()->set_right(previous_right);
    } else if (node->is_left_child()) {
      node->get_parent()->set_left(previous_right);
    } else if (node == this->get_root()) {
      this->set_root(previous_right);
      previous_right->set_parent(nullptr);
    }
    node->set_parent(previous_right);
    node->set_right(previous_right->get_left());
    previous_right->set_left(node);
  }
};
```

example usage:

```
#include <iostream>


int main() {
  BinarySearchTree<int> t;
  t.insert(11)->insert(5)->insert(3)->insert(4)->insert(2)->insert(1)->insert(7)->insert(6)
      ->insert(20)->insert(15)->insert(24)->insert(16)->insert(12)->insert(14)->insert(13)->
      insert(17)->insert(26);
  std::cout << "is this tree balanced?: " << t.is_balanced() << std::endl;
  std::cout << "the height of this tree is: " << t.get_root()->height() << std::endl;
  std::cout << "the balance factor of the root is: " << t.get_root()->balance_factor() <<
      std::endl;
  print_latex_forest(t.get_root());
  BinaryTreeNode<int>* newroot = t.get_root()->find_path(t.find(13));
  print_latex_forest(newroot);
  delete newroot;
}
```

is this tree balanced?: 0
the height of this tree is: 6
the balance factor of the root is: -1



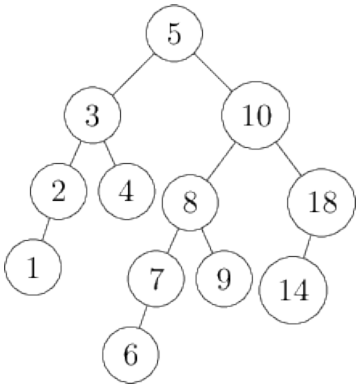


5.3.3 AVL tree

an **AVL tree** is a binary search tree in which every node `x` has a balance factor of $0, 1, -1$, meaning:

$$\left|\text{height}\left(T_{\text{left}(x)}\right) - \text{height}\left(T_{\text{right}(x)}\right)\right| \leq 1$$

every node `x` of this tree holds an extra field called `balance` which holds the balance factor of said node
for example, the following tree is an AVL tree:



if we were to remove the node with value 14 or with value 9 this wouldnt be an AVL tree

Lemma 1. let `T` be an AVL tree with `n` nodes and height h, then $h = \Theta(\log n)$

minimal form

Definition 5.3.9. a minimal form of an AVL tree with a given height h is one which contains the least possible number of nodes

Lemma 2. the minimal number of nodes for a tree of height h is given by the following recurrence relation

$$\begin{aligned} N(h) &= N(h-1) + N(h-2) + 1 \\ N(1) &= 1 \\ N(2) &= 2 \end{aligned}$$

> **note**
> according to some sources, the base cases are $N(0) = 1,\ N(1) = 2$, but the base cases given above make more sense because a tree with a single node should have a height of 1 not 0

Lemma 3. a minimal form of an AVL tree with a given number of nodes N is the one with the smallest height which is given by:

$$h = \log_2 N$$

here is a python snippet to find the minimal number of nodes needed for a given height:

```
def find(height):
  if height == 1:
    return 1
  if height == 2:
    return 2
  return find(height-1) + find(height-2) + 1
return [(i,find(i)) for i in range(1,15)]
```

height	nodes
1	1
2	2
3	4
4	7
5	12
6	20
7	33
8	54
9	88
10	143
11	232
12	376
13	609
14	986

problematic node

Definition 5.3.10. a **problematic node** is a node whose balance factor is ± 2

insertion into AVL tree the process of inserting a node to an AVL tree starts with inserting it using the same insertion method we used for a binary search tree, after said insertion we start going up the tree starting at the new node we just inserted and updating the balance factor for every node we encounter as we go
if we arrive at problematic node, we apply the proper rotation to rebalance the tree

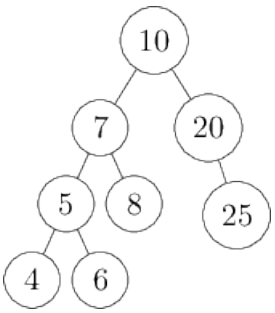
rebalancing during a modifying operation of a binary tree, a node might become problematic, to restore balance to the tree so that it remains an AVL tree we use rotations

rotation to determine which type of **rotation** we need to use to **rebalance** a given tree, we go 2 steps from the problematic node towards the newly inserted node which gives us the following 4 cases:

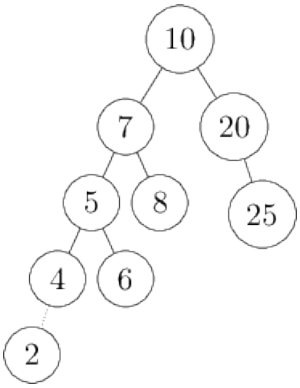
1. if we go 2 times to the left, then the **rotation** is of type LL
2. if we go 2 times to the right, then the **rotation** is of type RR
3. if we go left and then right then the **rotation** is of type LR
4. if we go right and then left then the **rotation** is of type RL

1. simple rotation
   (a) Left Left let `z` be the left child of the problematic node and `x` be the problematic node itself
   in **Left Left** rotation we rotate `x` to the right, such that the left child of `x` replaces `x` and `x` gets "rotated" to the right becoming the right child of `z` and a parent of the right child of `z`

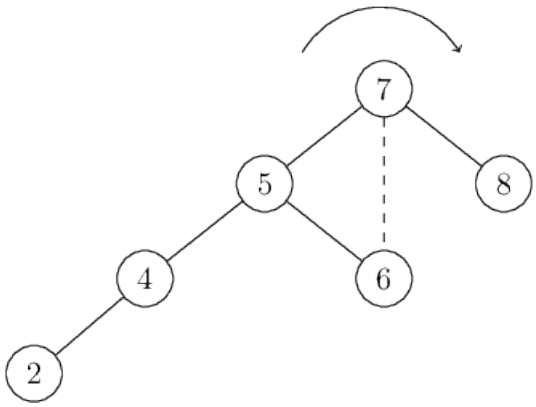
   Example 5.3.1. consider the following AVL tree:



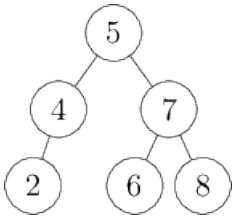
   after inserting a new node with the value 2 this is how the tree would look:



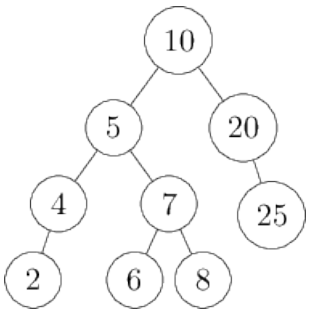
the node with the value 7 whose balance factor is 2 is problematic because its causing imbalance in the tree, so we apply the rotation to its subtree to rebalance it



the subtree after the rotation:



and so the original tree would become:



(b) Right Right this is very similar to Left Left, we first add the node and then if we have a problematic node `x` and if we took 2 steps to the right, then we use this rotation method
this method consists of rotating the subtree T_x such that `x` becomes a left child of its original right child `z`, and `z` takes the place of `x` and the previous left child of `z` becomes a right child of `x` at its new position

2. double rotation the different between this and simple rotation is that this method consists of rotating twice as you might've already guessed from the title

   (a) Left Right this method consists of 2 steps, first is applying an RR rotation over the left child of the problematic node, then applying an LL rotation to the problematic node

   (b) Right Left this method consists of 2 steps, first is applying an LL rotation over the right child of the problematic node, then applying a RR rotation to the problematic node

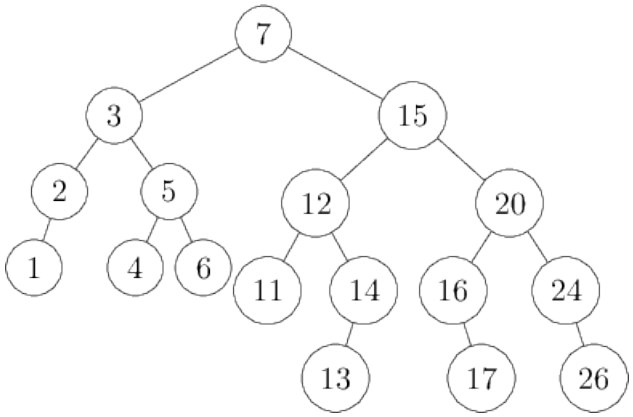
AVLTree

```
template <typename T> class AVLTree : public BinarySearchTree<T> {
public:
  using BinarySearchTree<T>::insert;
```

```
  AVLTree() {}
  AVLTree(BinaryTreeNode<T>* node) : BinarySearchTree<T>(node) {}
  AVLTree<T>* insert(BinaryTreeNode<T>* node) override {
    BinarySearchTree<T>::insert(node);
    BinaryTreeNode<T>* retracing_node = node;
    while (retracing_node != nullptr) {
      if (retracing_node->is_problematic()) {
        BinaryTreeNode<T>* path_root = retracing_node->find_path(node);
        if (path_root->has_left() && path_root->get_left()->has_left()) { /* LL rotation */
          this->rotate_right(retracing_node);
        }
        if (path_root->has_right() && path_root->get_right()->has_right()) { /* RR rotation
            */
          this->rotate_left(retracing_node);
        }
        if (path_root->has_right() && path_root->get_right()->has_left()) { /* RL rotation
            */
          this->rotate_right(retracing_node->get_right()); /* LL rotation */
          this->rotate_left(retracing_node); /* RR rotation */
        }
        if (path_root->has_left() && path_root->get_left()->has_right()) { /* LR rotation */
          this->rotate_left(retracing_node->get_left()); /* RR rotation */
          this->rotate_right(retracing_node); /* LL rotation */
        }
          delete path_root;
          break;
        }
      retracing_node = retracing_node->get_parent();
    }
    return this;
  }
};
```

example:

```
#include <iostream>



int main() {
  AVLTree<int> t;
  t.insert(11)->insert(5)->insert(3)->insert(4)->insert(2)->insert(1)->insert(7)->insert(6)
      ->insert(20)->insert(15)->insert(24)->insert(16)->insert(12)->insert(14)->insert(13)->
      insert(17)->insert(26);
  print_latex_forest(t.get_root());
}
```



5.3.4 2-3 tree

a **23 tree** is a tree data structure where every internal node is either a 2-node or 3-node
an exceptional case is when the tree contains only one element in which case the only node in the tree (which is the root) holds only one element

characteristics

- every internal node is a 2-node or a 3-node
- all leaves are at the same level
- keys are always ordered from left to right in ascending order

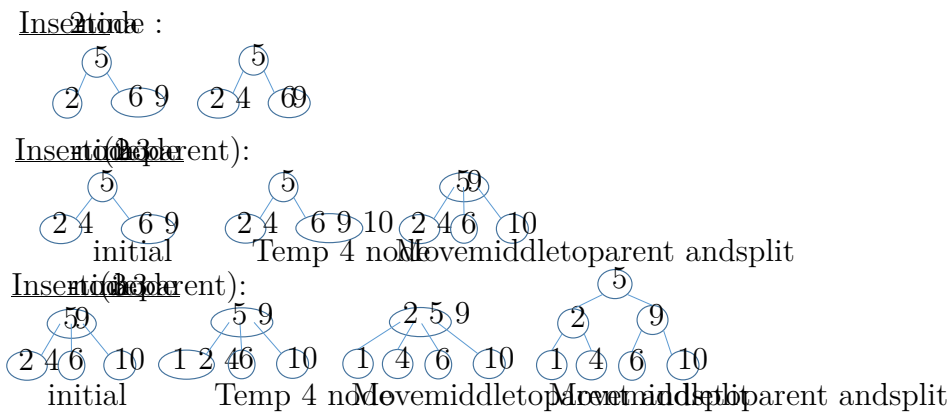
2-node

Definition 5.3.11. we say that an internal node is a **2-node** if it holds one value and two children

3-node

Definition 5.3.12. we say that an internal node is a **3-node** if it holds two values and three children

insertion an important property to keep in mind is that insertion **always** occurs in one of the leaves



TwoThreeNode

```
const int SIZE = 3;

template <typename T> class TwoThreeNode {
private:
  TwoThreeNode<T>* parent;
  TwoThreeNode<T>* children[SIZE+1];
  int keys[SIZE];
  T values[SIZE];
public:
  void set_child(int idx, TwoThreeNode<T>* node) {
    this->children[idx] = node;
    if (node)
      node->parent = this;
  }
  TwoThreeNode<T>* get_child(int idx) {
    return this->children[idx];
  }
  void remove_child(TwoThreeNode<T>* node) {
    for (int i = 0; i < SIZE; ++i) {
      if (this->children[i] == node) {
        this->children[i]->parent = nullptr;
        this->children[i] = nullptr;
        return;
      }
    }
  }
  int add_child(TwoThreeNode<T>* node) {
    for (int i = 0; i < SIZE; ++i) {
      if (this->keys[i] > node->biggest_key() || this->keys[i] == 0) {
        this->set_child(i, node);
        return i;
      }
    }
    this->set_child(SIZE, node);
    return SIZE;
  }
  int add_key_value(int key, T value) {
    for (int i = 0; i < SIZE; ++i) {
      if (this->keys[i] == 0) {
        this->keys[i] = key;
        this->values[i] = value;
        return i;
      }
      if (this->keys[i] > key) {
        this->children[SIZE] = this->children[SIZE-1];
        for (int j = SIZE-1; j > i; --j) {
          this->keys[j] = this->keys[j-1];
          this->values[j] = this->values[j-1];
          this->children[j] = this->children[j-1];
        }
        this->keys[i] = key;
        this->values[i] = value;
        return i;
      }
    }
```

```
  return -1;
}
void set_key(int idx, int key) {
  this->keys[idx] = key;
}
int get_key(int idx) {
  return this->keys[idx];
}
bool has_key(int idx) {
  return this->keys[idx] != 0;
}
bool has_child(int idx) {
  return this->children[idx] != nullptr;
}
void set_value(int idx, T value) {
  this->values[idx] = value;
}
T get_value(int idx) {
  return this->values[idx];
}
void set_parent(TwoThreeNode<T>* node) {
  this->parent = node;
}
TwoThreeNode<T>* get_parent() {
  return this->parent;
}
bool has_parent() {
  return this->parent != nullptr;
}
bool is_leaf() {
  for (int i = 0; i < SIZE; ++i) {
    if (children[i] != nullptr)
      return false;
  }
  return true;
}
bool is_root() {
  return !this->has_parent();
}
TwoThreeNode<T>* find_node_to_place_key(int key) {
  if (this->is_leaf()) {
    for (int i = 0; i < SIZE; ++i) {
      if (this->keys[i] > key || this->keys[i] == 0) {
        return this;
      }
    }
    return nullptr;
  }
  for (int i = 0; i < SIZE; ++i) {
    if (this->keys[i] > key || this->keys[i] == 0) {
      return this->children[i]->find_node_to_place_key(key);
    }
  }
  return this->children[SIZE] == nullptr ? nullptr : this->children[SIZE]->
      find_node_to_place_key(key);
}
```

```
TwoThreeNode<T>* find_node_with_biggest_key() {
  if (this->is_leaf())
    return this;
  TwoThreeNode<T>* biggest = nullptr;
  int biggest_key = 0;
  for (int i = 0; i < SIZE; ++i) {
    if (this->children[i] != nullptr) {
      TwoThreeNode<T>* tmp = this->children[i]->find_node_with_biggest_key();
      if (tmp == nullptr)
        continue;
      for (int j = 0; j < tmp->count_keys(); ++j) {
        if (tmp->keys[j] > biggest_key) {
          biggest = tmp;
          biggest_key = tmp->keys[j];
        }
      }
    }
  }
  return biggest;
}
int find_biggest_key() {
  if (this->is_leaf()) {
    return this->biggest_key();
  }
  int biggest = this->biggest_key();
  for (int i = 0; i < SIZE; ++i) {
    if (this->children[i] != nullptr) {
      int other = this->children[i]->find_biggest_key();
      if (other > biggest)
        biggest = other;
    }
  }
  return biggest;
}
int find_smallest_key() {
  if (this->is_leaf()) {
    return this->smallest_key();
  }
  int smallest = this->smallest_key();
  for (int i = 0; i < SIZE; ++i) {
    if (this->children[i] != nullptr) {
      int other = this->children[i]->find_smallest_key();
      if (other != 0 && other < smallest && other != -1)
        smallest = other;
    }
  }
  return smallest;
}
int biggest_key() {
  int biggest = 0;
  for (int i = 0; i < SIZE; ++i) {
    if (this->keys[i] > biggest)
      biggest = this->keys[i];
  }
  return biggest;
}
```

```
int smallest_key() {
  int smallest = -1;
  for (int i = 0; i < SIZE; ++i) {
    if ((this->keys[i] < smallest && this->keys[i] != 0) || smallest == -1)
      smallest = this->keys[i];
  }
  return smallest;
}
int count_keys() {
  int cnt = 0;
  for (int i = 0; i < SIZE; ++i) {
    if (this->keys[i] != 0) {
      ++cnt;
    }
  }
  return cnt;
}
int count_children() {
  int cnt = 0;
  for (int i = 0; i < SIZE; ++i) {
    if (this->children[i] != nullptr) {
      ++cnt;
    }
  }
  return cnt;
}
bool is_2_node() {
  return this->count_keys() == 1;
}
bool is_3_node() {
  return this->count_keys() == 2;
}
void print() {
  for (int i = 0; i < SIZE; ++i) {
    std::cout << this->keys[i] << " ";
  }
  std::cout << std::endl;
}
std::string to_string() {
  std::string str = "";
  for (int i = 0; i < SIZE; ++i) {
    std::string num_str = this->keys[i] == 0 ? "$\\varnothing$" : std::to_string(this->
        keys[i]);
    str += num_str + (i == SIZE - 1 ? "" : " ");
  }
  return str;
}
int height() {
  if (this->is_leaf())
    return 1;
  for (int i = 0; i < SIZE; ++i) {
    if (this->children[i] != nullptr) {
      return 1+this->children[i]->height();
    }
  }
  return 1;
```

```
  }
  int get_left_key() {
    return this->keys[0];
  }
  int get_right_key() {
    return this->keys[1];
  }
  bool has_left_key() {
    return this->keys[0] != 0;
  }
  bool has_right_key() {
    return this->keys[1] != 0;
  }
  TwoThreeNode<T>* get_left_child() {
    return this->children[0];
  }
  TwoThreeNode<T>* get_right_child() {
    return this->children[2];
  }
  TwoThreeNode<T>* get_mid_child() {
    return this->children[1];
  }
  bool has_left_child() {
    return this->children[0] != nullptr;
  }
  bool has_right_child() {
    return this->children[2] != nullptr;
  }
  bool has_mid_child() {
    return this->children[1] != nullptr;
  }
};
```

TwoThreeTree

```
template <typename T> class TwoThreeTree {
private:
  TwoThreeNode<T>* root = nullptr;
public:
  TwoThreeNode<T>* get_root() {
    return this->root;
  }
  void add_key_value(int key, T value) {
    if (this->root == nullptr) {
      this->root = new TwoThreeNode<T>();
      this->root->set_key(0, key);
      this->root->set_value(0, value);
      return;
    }
    TwoThreeNode<T>* node = this->root->find_node_to_place_key(key);
    if (node == nullptr) {
      node = this->root->find_node_with_biggest_key();
    }
    std::cout << "adding key " << key << " to " << (node == nullptr ? "unknown" : node->
        to_string()) << std::endl;
```

```
  int key_count = node->count_keys();
  if (key_count < SIZE-1) {
    std::cout << "added " << key << " to " << node->to_string() << " and got ";
    node->add_key_value(key, value);
    std::cout << node->to_string() << std::endl;
    return;
  }
  if (key_count >= SIZE-1) {
    node->add_key_value(key, value);
    std::cout << "balance " << node->to_string() << std::endl;
    balance(node);
  }
}
/* helping function */
void balance(TwoThreeNode<T>* node) {
  std::cout << "root: " << this->root->to_string() << std::endl;
  if (node->is_2_node() || node->is_3_node()) {
    std::cout << node->to_string() << " is good." << std::endl;
    return;
  }
  if (node->is_root()) {
    TwoThreeNode<T>* new_root = new TwoThreeNode<T>();
    new_root->add_key_value(node->get_key(1), node->get_value(1));
    TwoThreeNode<T>* new_left_node = new TwoThreeNode<T>();
    TwoThreeNode<T>* new_right_node = new TwoThreeNode<T>();
    new_left_node->set_key(0, node->get_key(0));
    new_right_node->set_key(0, node->get_key(2));
    new_left_node->set_child(0, node->get_child(0));
    new_left_node->set_child(1, node->get_child(1));
    new_right_node->set_child(0, node->get_child(2));
    new_right_node->set_child(1, node->get_child(3));
    new_root->set_child(0, new_left_node);
    new_root->set_child(1, new_right_node);
    std::cout << "created new root " << new_root->to_string() << " to replace " << this->
        root->to_string() << ", left child: " << new_left_node->to_string() << ", right
        child: " << new_right_node->to_string() << std::endl;
    this->root = new_root;
    return;
  }
  TwoThreeNode<T>* the_parent = node->get_parent();
  std::cout << "adding " << node->get_key(1) << " to " << the_parent->to_string() << " to
      get ";
  the_parent->add_key_value(node->get_key(1), node->get_value(1));
  std::cout << the_parent->to_string() << std::endl;
  node->set_key(1, 0);
  split(node);
  std::cout << "balance " << the_parent->to_string() << " recursively" << std::endl;
  balance(the_parent);
  delete node;
}
void split(TwoThreeNode<T>* node) {
  std::cout << "split " << node->to_string() << std::endl;
  TwoThreeNode<T>* the_parent = node->get_parent();
  TwoThreeNode<T>* new_left_node = new TwoThreeNode<T>();
  TwoThreeNode<T>* new_right_node = new TwoThreeNode<T>();
  new_left_node->set_key(0, node->get_key(0));
```

```
    new_right_node->set_key(0, node->get_key(2));
    new_left_node->set_child(0, node->get_child(0));
    new_left_node->set_child(1, node->get_child(1));
    new_right_node->set_child(0, node->get_child(2));
    new_right_node->set_child(1, node->get_child(3));
    the_parent->remove_child(node);
    std::cout << "inserting right " << new_right_node->to_string() << " as a " << the_parent
       ->add_child(new_right_node) << "th child of " << the_parent->to_string() << std::endl
       ;
    //the_parent->add_child(new_right_node);
    std::cout << "inserting left " << new_left_node->to_string() << " as a " << the_parent->
       add_child(new_left_node) << "th child of " << the_parent->to_string() << std::endl;
    //the_parent->add_child(new_left_node);
  }
  bool is_empty() {
    return this->root == nullptr;
  }
  int height() {
    return this->is_empty() ? 0 : this->root->height();
  }
};
```

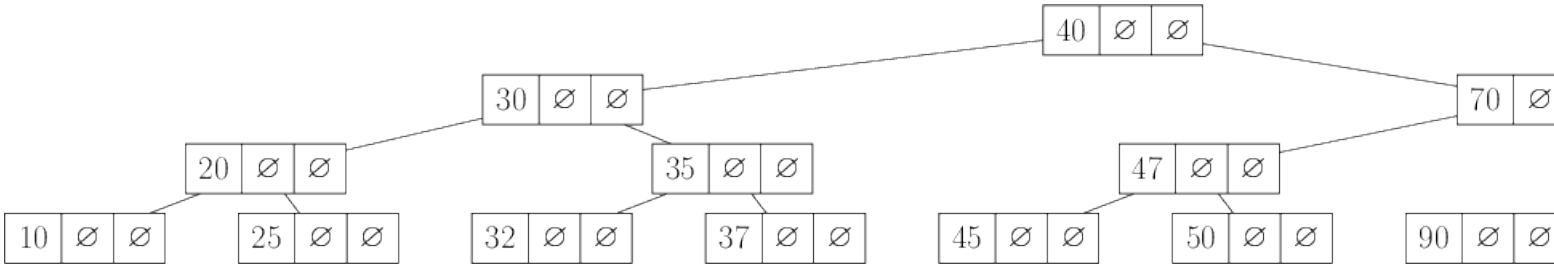
example usage:

```
#include <iostream>




int main() {
  TwoThreeTree<float> t;
  const float seq[] = {3,2,7,9,5,4,3.5,4.5,3.2,4.7,1,10,11,10.5,2.5,3.7,12};
  for (int i = 0; i < 17; ++i) {
    t.add_key_value((int)(seq[i]*10), seq[i]);
  }
  print_latex_forest_twothree_tree(t.get_root());
}
```

adding key 20 to 30 ∅ ∅
added 20 to 30 ∅ ∅ and got 20 30 ∅
adding key 70 to 20 30 ∅
balance 20 30 70
root: 20 30 70
created new root 30 ∅ ∅ to replace 20 30 70, left child: 20 ∅ ∅, right child: 70 ∅ ∅
adding key 90 to 70 ∅ ∅
added 90 to 70 ∅ ∅ and got 70 90 ∅
adding key 50 to 70 90 ∅
balance 50 70 90
root: 30 ∅ ∅
adding 70 to 30 ∅ ∅ to get 30 70 ∅
split 50 ∅ 90
inserting right 90 ∅ ∅ as a 2th child of 30 70 ∅
inserting left 50 ∅ ∅ as a 1th child of 30 70 ∅
balance 30 70 ∅ recursively
root: 30 70 ∅
30 70 ∅ is good.
adding key 40 to 50 ∅ ∅
added 40 to 50 ∅ ∅ and got 40 50 ∅

adding key 35 to 40 50 ∅
balance 35 40 50
root: 30 70 ∅
adding 40 to 30 70 ∅ to get 30 40 70
split 35 ∅ 50
inserting right 50 ∅ ∅ as a 2th child of 30 40 70
inserting left 35 ∅ ∅ as a 1th child of 30 40 70
balance 30 40 70 recursively
root: 30 40 70
created new root 40 ∅ ∅ to replace 30 40 70, left child: 30 ∅ ∅, right child: 70 ∅ ∅
adding key 45 to 50 ∅ ∅
added 45 to 50 ∅ ∅ and got 45 50 ∅
adding key 32 to 35 ∅ ∅
added 32 to 35 ∅ ∅ and got 32 35 ∅
adding key 47 to 45 50 ∅
balance 45 47 50
root: 40 ∅ ∅
adding 47 to 70 ∅ ∅ to get 47 70 ∅
split 45 ∅ 50
inserting right 50 ∅ ∅ as a 1th child of 47 70 ∅
inserting left 45 ∅ ∅ as a 0th child of 47 70 ∅
balance 47 70 ∅ recursively
root: 40 ∅ ∅
47 70 ∅ is good.
adding key 10 to 20 ∅ ∅
added 10 to 20 ∅ ∅ and got 10 20 ∅
adding key 100 to 90 ∅ ∅
added 100 to 90 ∅ ∅ and got 90 100 ∅
adding key 110 to 90 100 ∅
balance 90 100 110
root: 40 ∅ ∅
adding 100 to 47 70 ∅ to get 47 70 100
split 90 ∅ 110
inserting right 110 ∅ ∅ as a 3th child of 47 70 100
inserting left 90 ∅ ∅ as a 2th child of 47 70 100
balance 47 70 100 recursively
root: 40 ∅ ∅
adding 70 to 40 ∅ ∅ to get 40 70 ∅
split 47 ∅ 100
inserting right 100 ∅ ∅ as a 2th child of 40 70 ∅
inserting left 47 ∅ ∅ as a 1th child of 40 70 ∅
balance 40 70 ∅ recursively
root: 40 70 ∅
40 70 ∅ is good.
adding key 105 to 110 ∅ ∅
added 105 to 110 ∅ ∅ and got 105 110 ∅
adding key 25 to 10 20 ∅
balance 10 20 25
root: 40 70 ∅
adding 20 to 30 ∅ ∅ to get 20 30 ∅
split 10 ∅ 25
inserting right 25 ∅ ∅ as a 1th child of 20 30 ∅
inserting left 10 ∅ ∅ as a 0th child of 20 30 ∅
balance 20 30 ∅ recursively
root: 40 70 ∅
20 30 ∅ is good.

adding key 37 to 32 35 ∅
balance 32 35 37
root: 40 70 ∅
adding 35 to 20 30 ∅ to get 20 30 35
split 32 ∅ 37
inserting right 37 ∅ ∅ as a 3th child of 20 30 35
inserting left 32 ∅ ∅ as a 2th child of 20 30 35
balance 20 30 35 recursively
root: 40 70 ∅
adding 30 to 40 70 ∅ to get 30 40 70
split 20 ∅ 35
inserting right 35 ∅ ∅ as a 1th child of 30 40 70
inserting left 20 ∅ ∅ as a 0th child of 30 40 70
balance 30 40 70 recursively
root: 30 40 70
created new root 40 ∅ ∅ to replace 30 40 70, left child: 30 ∅ ∅, right child: 70 ∅ ∅
adding key 120 to 105 110 ∅
balance 105 110 120
root: 40 ∅ ∅
adding 110 to 100 ∅ ∅ to get 100 110 ∅
split 105 ∅ 120
inserting right 120 ∅ ∅ as a 2th child of 100 110 ∅
inserting left 105 ∅ ∅ as a 1th child of 100 110 ∅
balance 100 110 ∅ recursively
root: 40 ∅ ∅
100 110 ∅ is good.



5.3.5 algorithms

backtracking **Backtracking** is an algorithm that incrementally builds candidates to the solutions, and abandons a candidate as soon as it determines that the candidate cannot possibly be completed to a valid solution, see `https://en.wikipedia.org/wiki/Backtracking`

pattern this includes the backtracking algorithm
after solving a few problems i saw a pattern that makes it much easier to solve recursion problems related to binary trees
when i want to solve a problem with recursion i ask myself 3 questions

1. what is a node's job?

2. what info does a node require to do its job?

3. how do we give the node what it requires to do its job? (one of 2 methods)

the 2 methods of passing data around binary tree nodes are:

- if a node needs to know some info about its ancestors or nodes from other subtrees this info has to be passed through function arguments because once we reach a node in the tree we cant go back to previous nodes (its possible but would take `O(n)` time which is horrible)

- if a node needs to know something about its descendants, this information has to be backpropagated by returning it to the parent node that called the recursive function on its child, because once we're back to a node we cant go back down (we could but again, it takes `O(n)` time), if we need to backpropagate multiple values we can use a simple structure to hold the values (in `C++` the simplest way is to use `struct`)

note that its **important** that we gather info as we go on every node we visit so that once we leave a specific node we'd have all the info we would need from it so that we wouldnt have to go back to visit it which usually results in `O(n)` time
words arent the best way to put it so hopefully ill provide a better visual explanation in the future

reverse inorder given a binary tree, write a function to reverse its inorder traversal output
this algorithm runs in `O(n)` time

```
template <typename T>
void reverse(BinaryTreeNode<T>* node) {
  if (node == nullptr)
    return;
  reverse(node->get_left());
  reverse(node->get_right());
  BinaryTreeNode<T>* l = node->get_left();
  node->set_left(node->get_right());
  node->set_right(l);
}
```

example usage:

```
#include <iostream>




int main() {
  BinarySearchTree<int> t;
  t.insert(11)->insert(5)->insert(3)->insert(4)->insert(2)->insert(1)->insert(7)->insert(6)
     ->insert(20)->insert(15);
  t.print();
  reverse(t.get_root());
  t.print();
}
```

5.4 Heap

Definition 5.4.1. a Heap is defined as a data structure in which the elements exist in an almost complete tree
a heap may keep a pointer to speed up insertion, this pointer points to the leftmost leaf if the tree is complete, otherwise if its almost complete, it points to the right uncle of the rightmost leaf

note
in a heap the keys arent necessarily sorted like in binary search trees

5.4.1 min heap

Definition 5.4.2. a **min heap** is a Heap in which each node is less than or equal to both its children

5.4.2 max heap

Definition 5.4.3. a **max heap** is a Heap in which each node is bigger than or equal to both its children

this is the implementation we'll be using

5.4.3 binary heap

Definition 5.4.4. A **binary heap** is defined as a binary tree with two additional constraints:

1. shape property: a binary heap is an almost complete tree

2. heap property: the key stored in each node is either greater than or equal to, or less than or equal to the keys in the node's children

bubbling

Definition 5.4.5. moving a node up/down a heap to get it to its correct position

bubble up bubbling a node upwards consists of recursively switching it with its parent until its parent is bigger/smaller than it (depending on whether we're using a min heap or a max heap)

bubble down bubbling a node downards consists of recursively switching it with one of its children until it gets to its correct position

insertion to binary heap inserting a new node to the heap consists of adding it as a leaf such that the heap remains almost complete and applying bubble up to the newly added node so that the keys in the tree remain sorted

deletion from binary heap deletion of a node consists of replacing the rightmost leaf with it and bubbling it

binary heap array representation

note
some resources may use assume that arrays start at index 1, i see no point in that since most programming languages use 0 as the first index, so some of the following propositions might seem a little different like for example the proposition number 2 in the following list according to these sources would be that the

left child of ith node is $Arr[2i]$ instead of $Arr[2i+1]$

for an array to represent a heap, the only condition is that $A[i] < A\left[\left\lfloor \frac{i}{2} \right\rfloor\right]$
arrays that represent heap have the following characterstics:

1. root is $Arr[0]$
2. ith node is $Arr[\frac{i-1}{2}]$
3. left child of ith node is $Arr[2i+1]$
4. right child of ith node is $Arr[2i+2]$

in other words, the nodes are taken from the root to the last level from left to right and put in that order in an array
the array should hold 2 values, `array-size`, which is the size of the array, and `heap-size`, which is the number of elements of the heap in the array

Lemma 4. the left child of `A[i]` is `A[2i]` when $2i+1 \leq$ heap-size or `NULL` when $2i >$ heap-size
the right child of `A[i]` is `A[2i+1]` when $2i+1 \leq$ heap-size or `NULL` when $2i+1 >$ heap-size
the parent of `A[i]` is $A\left[\left\lfloor \frac{i}{2} \right\rfloor\right]$ when $i > 1$ or `NULL` when $i = 1$

we say in short that `A[i]` is bigger than `A[j]` instead of saying that the key of the `A[i]` is bigger than the key of `A[j]`

Lemma 5. since in a heap the key of a parent is bigger than the key of its children then in an array that contains a heap, for every $i > 1$, $A[i] < A\left[\frac{i}{2}\right]$

construction of binary heap from array given some random array, e.g. `A=[10,30,40,50,70,35,60,80]`, we need to reorganize its elements such that it represents a heap
to achieve this we apply bubbling to the first half of the array **in reverse**
e.g. for the given array we apply `bubble(A, i)` such that $i = 3 \to 0$
this is the constructing in pseudocode:

```
Build-Heap(A)
  n = length(A)
  heap-size = n
  for i = n/2 to 1
    heapify(A, i)
```

Lemma 6. this algorithm runs in $\Theta(n)$ time

Proof. for $0 < k < 1$ we know:

$$\sum_{x=1}^{\infty} x \cdot k^x = \frac{k}{(1-k)^2}$$

so for $k = \frac{1}{2}$:

$$\sum_{x=1}^{\infty} x \cdot \left(\frac{1}{2}\right)^x = \frac{1}{2} + \frac{2}{2^2} + \frac{3}{2^3} + \frac{4}{2^4} + \cdots = \frac{\frac{1}{2}}{\left(1 - \frac{1}{2}\right)^2} = \frac{\frac{1}{2}}{\frac{1}{4}} = 2$$

to be continued

$\square$

finding k biggest nodes given an array representing a heap A, we want an to print the k biggest nodes in A
first solution (inefficient) would be to repeatedly print the root of the heap and deleting it k times, this methods time complexity is $O(k \log n)$

second solution (more efficient) would be to create another Heap B and initiate it to contain the root of A and then repeatedly printing the root of B and deleting it and then adding its 2 children from A to B, which runs in $\Theta(k \log k)$ time

KeyValue

```
template <typename K>
template <typename V>
class KeyValue<K, V> {
private:
  K key;
  V value;
public:
  KeyValue(K key, V value) {
    this->key = key;
    this->value = value;
  }
  K get_key() {
    return this->key();
  }
  V get_value() {
    return this->value();
  }
  void set_value(V value) {
    this->value = value;
  }
  void set_key(K key) {
    this->key = key;
  }
};
```

BinaryHeap

```
template <typename T>
class BinaryHeap : public BinaryTree<T> {
public:
  BinaryHeap(BinaryTree<T>* root) : BinaryTree<T>(root) {}
  BinaryHeap() {}
  BinaryHeap<T>* insert(BinaryTreeNode<T>* node) {
    std::cout << "inserting " << node->get_key() << std::endl;
    if (BinaryTree<T>::is_empty()) {
      std::cout << "set root: " << node->get_key() << std::endl;
```

```
      BinaryTree<T>::set_root(node);
    } else {
      if (BinaryTree<T>::is_complete()) {
        std::cout << "tree complete, adding " << node->get_key() << " to the left" << std::
            endl;
        BinaryTree<T>::insert_left(node);
      } else { /* almost complete */
        BinaryTreeNode<T>* leaf = this->find_rightmost_leaf();
        if (leaf->is_left()) {
          std::cout << "tree almost complete, adding " << node->get_key() << " as a right
              child of " << leaf->get_parent()->get_key() << std::endl;
          leaf->get_parent()->set_right(node);
        } else {
          std::cout << "adding " << node->get_key() << " to right uncle of " << leaf->get_key
              () << " which is " << this->find_leftmost_leaf()->get_key() << std::endl;
          this->find_leftmost_leaf()->set_left(node);
        }
      }
    }
    return this;
  }
  BinaryHeap<T>* insert(int key) {
    return this->insert(new BinaryTreeNode<T>(key));
  }
};
```

```
template <typename T>
class BinaryHeap {
private:
  ArrayList<KeyValue<int, T>> arr;

public:
  void insert(KeyValue<) {
  }
};
```

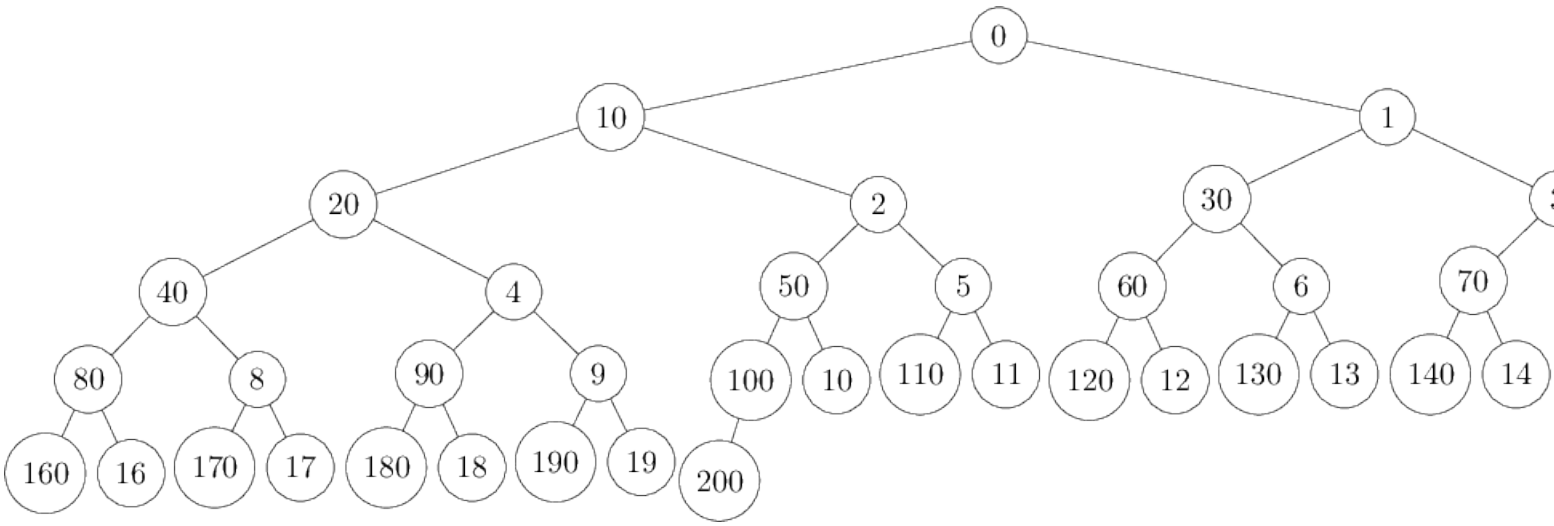
example usage:

```
#include <iostream>



int main() {
  BinaryHeap<int> h;
  for (int i = 0; i < 20; ++i) {
    h.insert(i);
    h.insert((i+1)*10);
  }
  print_latex_forest(h.get_root());
}
```

inserting 0
set root: 0
inserting 10

```
tree complete, adding 10 to the left
inserting 1
tree almost complete, adding 1 as a right child of 0
inserting 20
tree complete, adding 20 to the left
inserting 2
tree almost complete, adding 2 as a right child of 10
inserting 30
adding 30 to right uncle of 2 which is 1
inserting 3
tree almost complete, adding 3 as a right child of 1
inserting 40
tree complete, adding 40 to the left
inserting 4
tree almost complete, adding 4 as a right child of 20
inserting 50
adding 50 to right uncle of 4 which is 2
inserting 5
tree almost complete, adding 5 as a right child of 2
inserting 60
adding 60 to right uncle of 5 which is 30
inserting 6
tree almost complete, adding 6 as a right child of 30
inserting 70
adding 70 to right uncle of 6 which is 3
inserting 7
tree almost complete, adding 7 as a right child of 3
inserting 80
tree complete, adding 80 to the left
inserting 8
tree almost complete, adding 8 as a right child of 40
inserting 90
adding 90 to right uncle of 8 which is 4
inserting 9
tree almost complete, adding 9 as a right child of 4
inserting 100
adding 100 to right uncle of 9 which is 50
inserting 10
tree almost complete, adding 10 as a right child of 50
inserting 110
adding 110 to right uncle of 10 which is 5
inserting 11
tree almost complete, adding 11 as a right child of 5
inserting 120
adding 120 to right uncle of 11 which is 60
inserting 12
tree almost complete, adding 12 as a right child of 60
inserting 130
adding 130 to right uncle of 12 which is 6
inserting 13
tree almost complete, adding 13 as a right child of 6
inserting 140
adding 140 to right uncle of 13 which is 70
inserting 14
tree almost complete, adding 14 as a right child of 70
inserting 150
```

adding 150 to right uncle of 14 which is 7
inserting 15
tree almost complete, adding 15 as a right child of 7
inserting 160
tree complete, adding 160 to the left
inserting 16
tree almost complete, adding 16 as a right child of 80
inserting 170
adding 170 to right uncle of 16 which is 8
inserting 17
tree almost complete, adding 17 as a right child of 8
inserting 180
adding 180 to right uncle of 17 which is 90
inserting 18
tree almost complete, adding 18 as a right child of 90
inserting 190
adding 190 to right uncle of 18 which is 9
inserting 19
tree almost complete, adding 19 as a right child of 9
inserting 200
adding 200 to right uncle of 19 which is 100



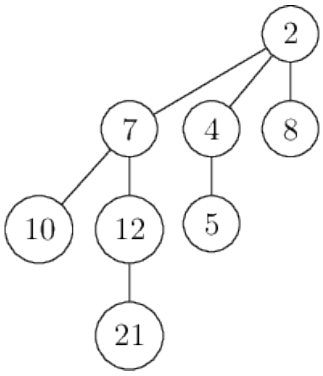
5.4.4 binomial heap

the explanation here is insufficient, see `https://en.wikipedia.org/wiki/Binomial_heap`

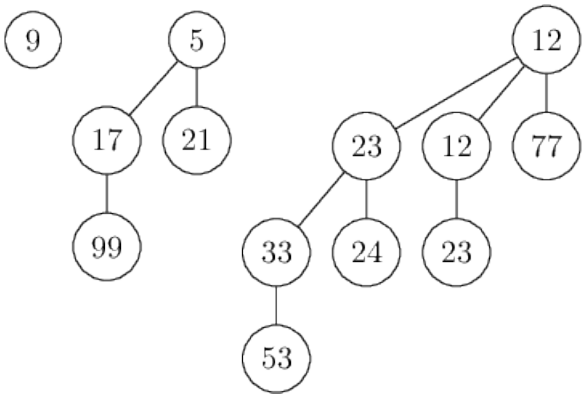
Definition 5.4.6. A **binomial heap** is implemented as a set of binomial trees (compare with a binary heap, which has a shape of a single binary tree), which are defined recursively as follows:

- A binomial tree of order 0 is a single node
- A binomial tree of order k has a root node whose children are roots of binomial trees of orders , k-1, $k-2$, $\ldots$, 2, 1, 0 (in this order)

Example 5.4.1. a binomial heap of order 3:



Example 5.4.2. this is an example binomial heap that consists of 13 nodes and 3 binomial trees



merging merging two binomial trees of the same order consists of simply adding the one with the smaller root as the last node of the other

5.4.5 algorithms

check if array represents a heap

```
is_heap(A, i, n)
  if i > n
    return true
  if (2i <= n and A[2i+1] > A[i]) or (2i+2 <= n and A[2i+2] > A[i])
    return false
  return is_heap(A, 2i+1, n) and is_heap(A, 2i+2, n)
```

heapify (bubble down)

```
l = left(i)
r = right(i)
if l <= heap-size[A] and A[l] > A[i]
  largest = l
else
  largest = i
if r <= heap-size[A] and A[r] > A[largest]
  largest = r
if largest != i
  exchange A[i] A[largest]
  heapify(A, largest)
```

time complexity is $O(\log n)$

5.5 Hash table

our goal is to find a data structure that is practical for inserting and retrieving values with time complexity that is as small as possible
the first solution would be to use an array where each index is a key and its corresponding cell holds the object whose key is that index, for example if the key of an object is 111, then $A[111]$ should give us the object
the second solution is to use a **Hash table** using the **chaining** method or the **open addressing** method
here we'll be using the chaining method

5.6 theorems

Lemma 7. the number of nodes in a complete tree of height h is $n = 2^h - 1$

Proof. we add all the nodes in a given complete tree according to their levels to get $n = 1+2+2^2+\cdots+2^{h-1} = 2^h - 1$

□

Lemma 8. the height of a complete tree with n nodes is $\log_2(n+1)$

Proof. according to the previous lemma $n+1 = 2^h$, therefore $h = \log_2(n+1)$

□

Lemma 9. the number of leaves in a complete tree with n nodes and height h is 2^{h-1}

Proof. using
the base case is $h = 1$, a complete tree of height 1 contains 1 node only which is a leaf, so $2^{h-1} = 2^{1-1} = 2^0 = 1$
we assume the statement holds true for every $h \leq i$
we prove the statements holds true for $h = i+1$, looking at a complete tree of height $i+1$, we can build it using a tree of height i by adding 2 children to every leaf, so $2 \cdot 2^{i-1} = 2^i$ which means the statement holds true for $i+1$

□

Lemma 10. the number of leaves in a complete tree that holds n nodes is $\frac{n+1}{2}$

Proof. we denote by h the height of the tree:

$$\text{number of leaves} = 2^{h-1} = 2^{\log_2(n+1)-1} = \frac{2^{\log_2(n+1)}}{2} = \frac{n+1}{2}$$

□

Lemma 11. in an almost complete tree, the number of nodes in a specific level is 2 times the number of nodes in level before it (except for perhaps the last level)

Lemma 12. the height of an almost complete tree with n nodes is $\lfloor \log_2 n \rfloor$

Lemma 13. the number of leaves in an almost complete tree with n nodes is $\left\lceil \frac{n}{2} \right\rceil$

Lemma 14. the number of nodes with height i in an almost complete tree with n nodes is $\left\lceil \frac{n}{2^i+1} \right\rceil$

Lemma 15. for every almost almost complete treecomplete tree with n nodes and height h, $2^{h-1} \leq n \leq 2^h - 1$

Proof. the biggest almost complete tree of height h is a complete tree and so $n \leq$ nodes in complete tree of height h $2^h - 1$
the smallest almost complete tree of height h can be constructed by adding a left child to the leftmost leaf of a complete tree of height $h-1$, $n \geq$ number of nodes in tree of height h-1 plus one $= 2^{h-1} - 1 + 1 = 2^{h-1}$ □

Lemma 16. for every almost complete tree with n nodes and height h, $\log_2(n+1) \leq h \leq 1 + \log_2 n$

Lemma 17. for every almost complete tree with n nodes and height h, $2^{h-2} \leq$ number of leaves $\leq 2^{h-1}$

5.7 time complexity table

> **note**
> this table assumes average case time complexity

data structure	insert	remove	search
doubly linked list with tail (given pointer)	$O(1)$	$O(1)$	$O(n)$
linked list without tail	$O(n)$	$O(n)$	$O(n)$
binary search tree	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap	$O(\log n)$	$O(\log n)$	$O(\log n)$
Hash table	$O(1)$	$O(1)$	$O(1)$
array list	$O(1)$	$O(n)$	$O(n)$

6 Sorting algorithms

6.1 Selection sort

Selection sort maintains two subarrays in a given array, one subarray that is sorted and the other unsorted, it sorts a given array by repeatedly finding the minimum element (considering ascending order) from the unsorted part and putting it at the beginning
in every iteration of selection sort, the minimum element (considering ascending order) from the unsorted subarray is picked and moved to the sorted subarray.
time complexity is $\Theta(n^2)$

```
template <typename T>
void selection_sort(T arr[], int len) {
  for (int i = 0; i < len; ++i) {
    // find index of minimum in unsorted subarray
    int min_idx = i;
    for (int j = i+1; j < len; ++j) {
      if (arr[j] < arr[min_idx]) {
        min_idx = j;
      }
    }

    // add minimum to the sorted subarray
    T tmp = arr[i];
    arr[i] = arr[min_idx];
    arr[min_idx] = tmp;
  }
}
```

example usage:

6.2 Insertion sort

1. iterate from arr[1] to arr[n]

2. compare the current element to its predecessor

3. if the current element is smaller than its predecessor, move it backwards until its bigger than its predecessor

time complexity is $O(n^2)$

```
template <typename T>
void insertion_sort(T arr[], int len) {
  for (int i = 1; i < len; ++i) {
    int idx = i;
    while (idx > 0 && arr[idx] < arr[idx-1]) {
      T tmp = arr[idx];
      arr[idx] = arr[idx-1];
      arr[idx-1] = tmp;
      idx--;
    }
  }
}
```

6.3 AVL sort

AVL sort consists of inserting the given array into an AVL tree element by element and then converting the tree into an array using inorder traversal

Lemma 18. the time complexity of this is $\Theta(n \log n)$

6.4 Heap sort

Heap sort consists of transforming the given array into a heap array `A`, initiating `heap-size` to `size(A)` then applying the following steps recursively:

1. replace the root of the heap `A[0]` with `A[heap-size - 1]`

2. bubble the newly replaced root

3. decrement `heap-size`, return to step 1 if `heap-size > 1`

Lemma 19. the time complexity of this is $\Theta(n \log n)$

6.5 QuickSort

QuickSort is a , it picks an element as **pivot** and **partitions** the given array around the pivot, there are different versions of quicksort that pick pivots in different ways:

1. first element as pivot (implemented here)

2. last element as pivot

3. a random element as pivot

4. median as pivot

the first subarray we use as a "partition" is the array itself, where given a pivot, put the pivot at its correct position in the sorted array by putting all the smaller elements before it, and all the greater elements after it. then divide the partition in 2 partitions, where the first partition is where all the smaller elements were put, and the second partition is where all the greater elements were put, and apply the same process for these partitions recursively until we reach partitions that contain 1 element only

this algorithm runs in $O(n \log n)$ time

```
template <typename T>
void swap(T arr[], int i, int j) {
  T tmp = arr[i];
  arr[i] = arr[j];
  arr[j] = tmp;
}

template <typename T>
int partition(T arr[], int low, int high) {
  int pivot = low++;
  while (low <= high) {
    if (arr[low] <= arr[pivot]) low++;
    else if (arr[high] >= arr[pivot]) high--;
    else swap(arr, low, high);
  }
  swap(arr, high, pivot);
  return high;
}

template <typename T>
void quick_sort(T arr[], int low, int high) {
  if (low <= high) {
    int pivot = partition(arr, low, high);
    quick_sort(arr, low, pivot-1);
    quick_sort(arr, pivot+1, high);
  }
}
```

6.6 Merge sort

yet another
given an array, this algorithm keeps splitting it until it reaches a subarray that cant be divided which happens when a subarray contains only 1 or 0 elements, each of these subarrays are sorted individually and then combined, recursively, to eventually make a larger sorted array
this algorithm runs in $\Theta(n \log n)$ time
`./merge_sort.webp`

the image (which visualizes the process) was taken from `https://www.programiz.com/dsa/merge-sort`, will keep this here until i make my own

```
#include <iostream>
template <typename T>
void merge(T arr[], int l, int m, int r) {
  int n1 = m - l + 1;
  int n2 = r - m;
  T L[n1];
  T R[n2];

  for (int i = 0; i < n1; ++i)
    L[i] = arr[l + i];
  for (int j = 0; j < n2; ++j)
    R[j] = arr[m + 1 + j];

  int i = 0, j = 0;
  int k = l;
  while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
      arr[k] = L[i];
      i++;
    }
    else {
      arr[k] = R[j];
      j++;
    }
    k++;
  }

  while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
  }
  while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
  }
}

template <typename T>
void sort(T arr[], int l, int r) {
  if (l < r) {
    int m = l+(r-l)/2;
    sort(arr, l, m);
    sort(arr, m + 1, r);
    merge(arr, l, m, r);
  }
}
```

7 Searching algorithms

7.1 binary search

goal: given a sorted array and a key, find index of the key in the array
method: compare key against middle entry

- too small, go left
- too big, go right
- equal, found

time complexity is $O(\log n)$

```
public static void main(String[] args) {
  int[] a = {1,10,11,21,30};
  System.out.println(binarySearch(a, 10));
}

static int binarySearch(int[] a, int key) {
  int lo = 0, hi = a.length - 1;
  while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (key < a[mid]) hi = mid - 1;
    else if (key > a[mid]) lo = mid + 1;
    else return mid;
  }
  return -1;
}
```

1

7.2 Ternary search